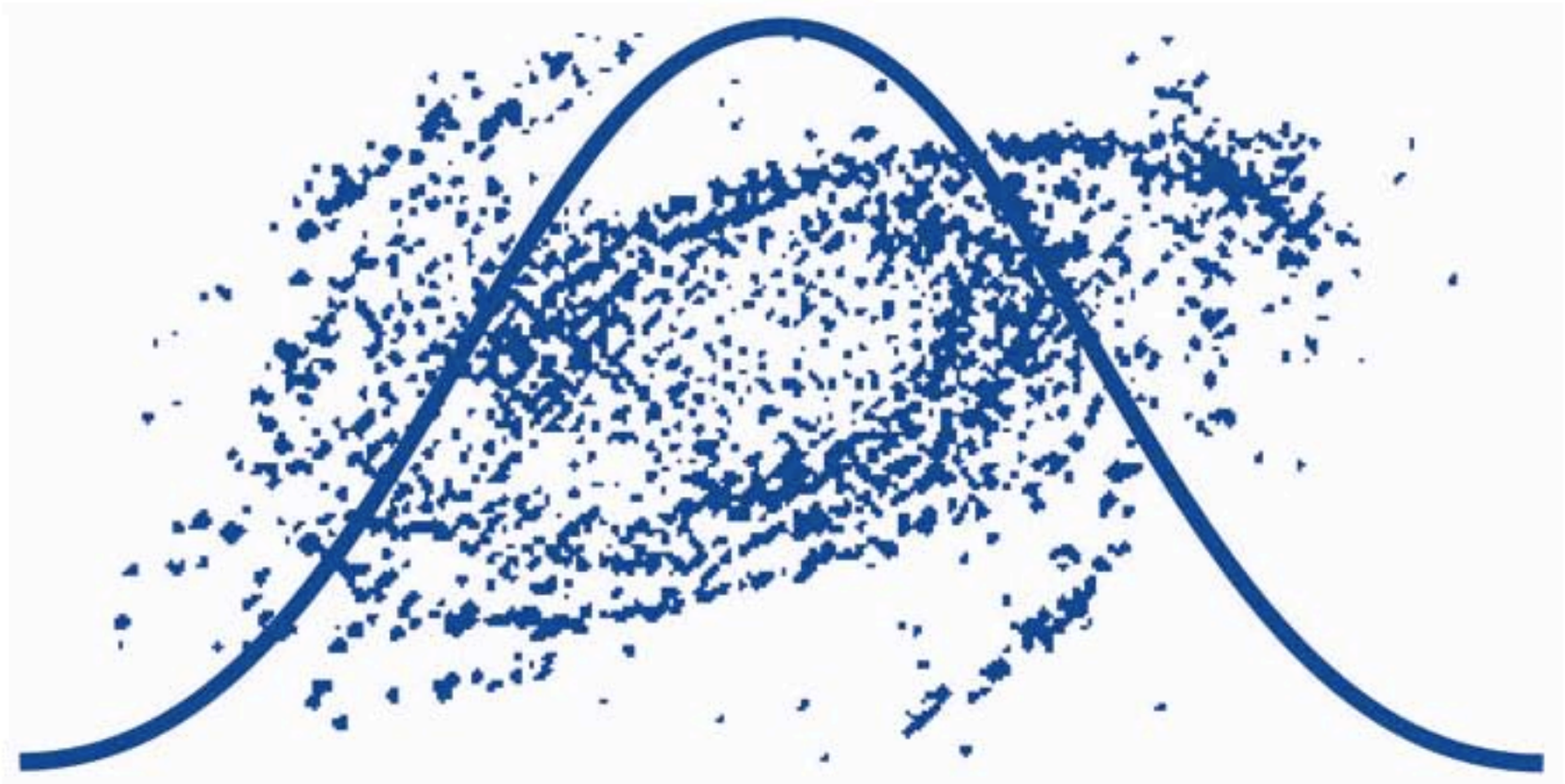
L8: Periodograms

ASTR3986: Astrostatistics

L8.1 Fourier



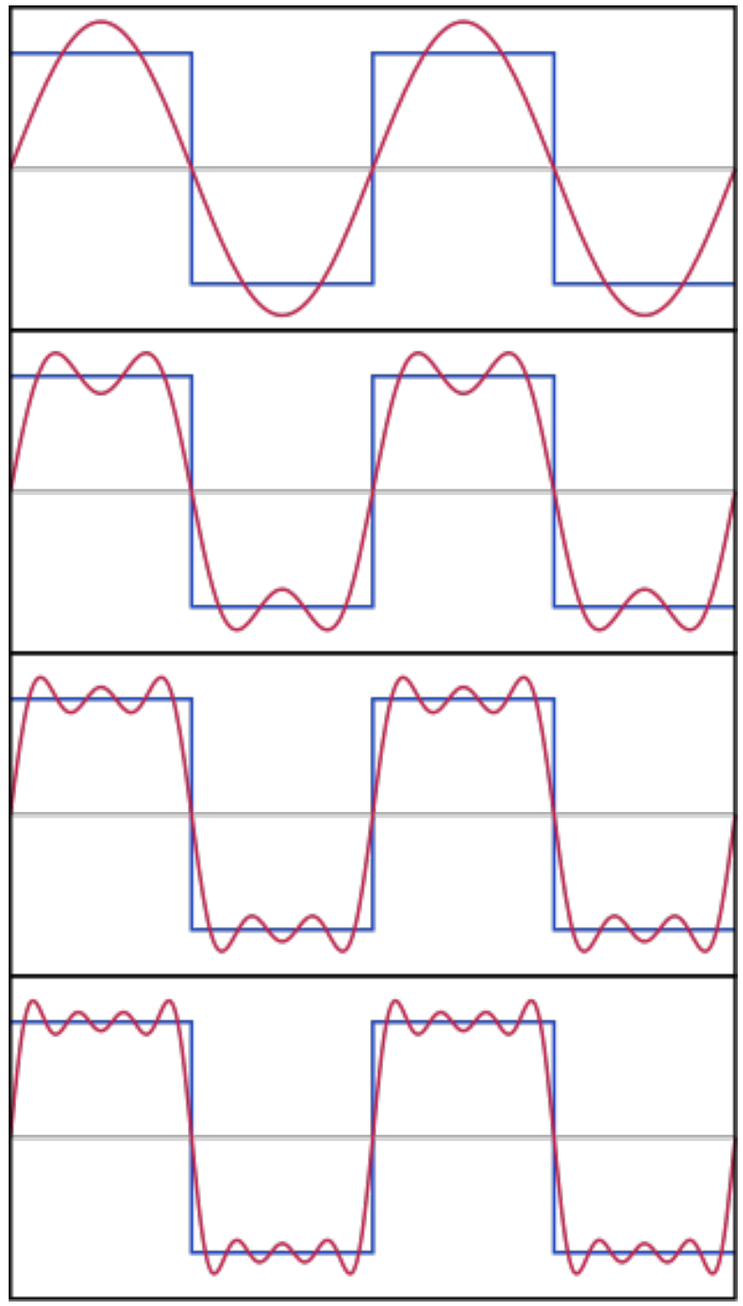
Fourier series

Joseph Fourier
(1768-1830)

Fourier series

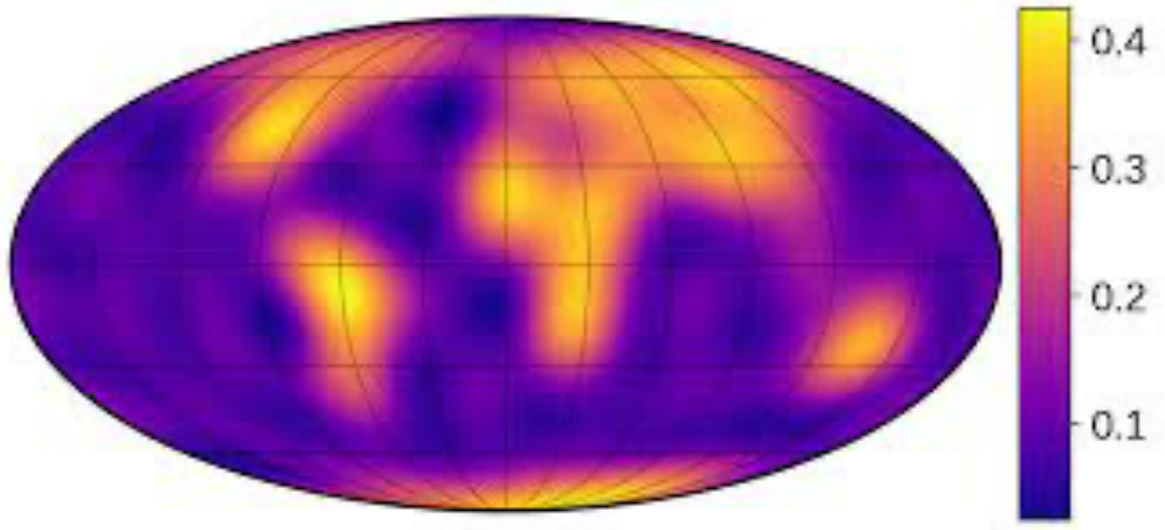
$$f(t) = \frac{A_0}{2} + \sum_{n=1}^{N} A_n \sin\left(\frac{2\pi n t}{P} + \phi_n\right)$$

the more terms you use,
the better the approximation



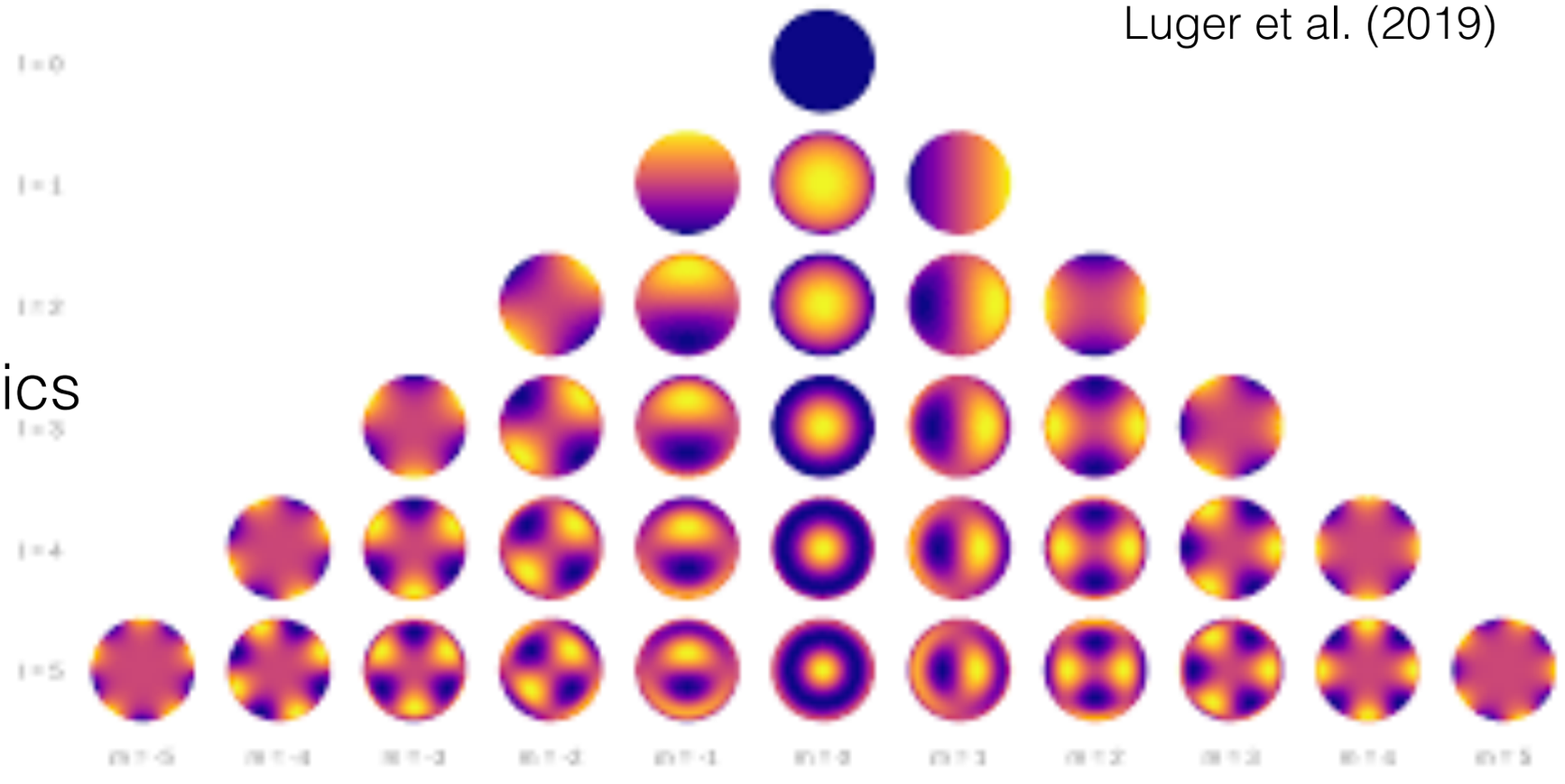
any periodic function can be written as a infinite sum of sine waves

Joseph Fourier
(1768-1830)



Luger et al. (2019)

spherical harmonics



Fourier series

$$f(t) = \frac{A_0}{2} + \sum_{n=1}^{N} A_n \sin\left(\frac{2\pi n t}{P} + \phi_n\right)$$

can also be written as

$$f(t) = \frac{a_0}{2} + \sum_{n=1}^{N} a_n \cos\left(\frac{2\pi n t}{P}\right) + b_n \sin\left(\frac{2\pi n t}{P}\right)$$

where...

$A_n = \sqrt{a_n^2 + b_n^2}$ *amplitude = modulus*

$\phi_n = \tan^{-1}(a_n/b_n)$ *phase = argument*

or, can also be written as

$$f(t) = \sum_{n=-N}^{N} c_n \exp\left(\mathrm{i}\frac{2\pi n t}{P}\right)$$

where we can solve for each coefficient using (Fourier proved this)

$$c_n = \frac{1}{P}\int_{-P/2}^{P/2} f(t)\exp(-2\pi \mathrm{i} t\omega/P)\,\mathrm{d}t$$

Fourier transform

Fourier transform

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t) \exp(-2\pi \mathrm{i} t\omega)\, \mathrm{d}t$$

the FT is complex-valued function in frequency

side note: relationship between Fourier series & Fourier transform

recall that the Fourier series coefficients are given by...

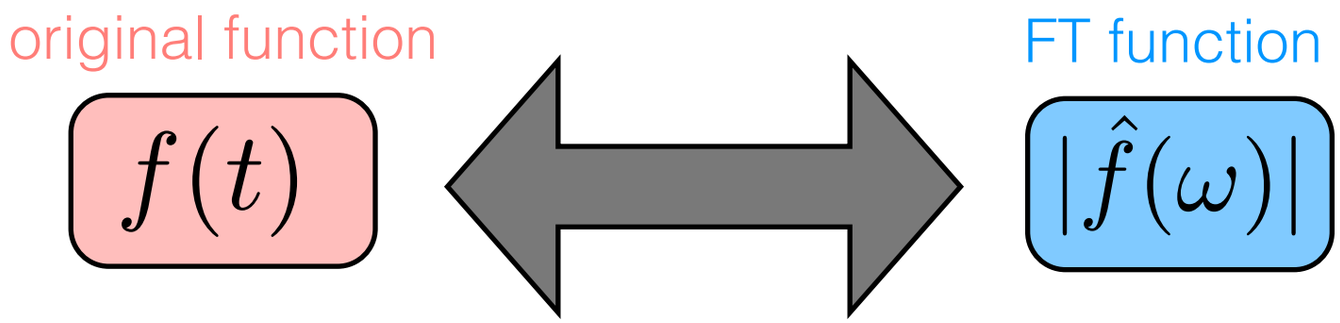
$$c_n = \frac{1}{P} \int_{-P/2}^{P/2} f(t) \exp(-2\pi \mathrm{i} t\omega/P)\, \mathrm{d}t$$

so, if we have the FT, we just write down these coeffs as...

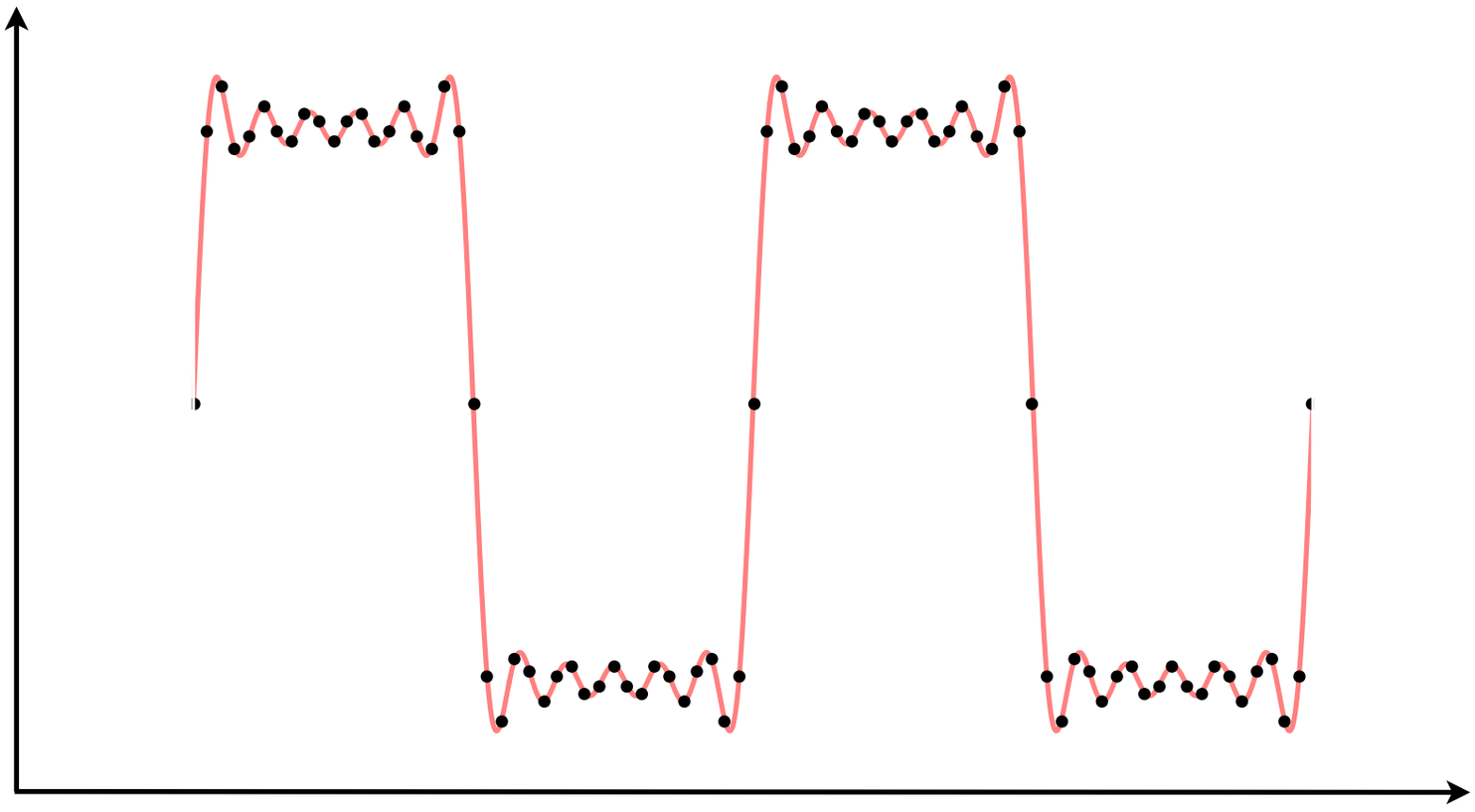
$$c_n = \frac{1}{P} \hat{f}(n/P)$$

complex modulus of the FT gives the amount of each frequency comprising f

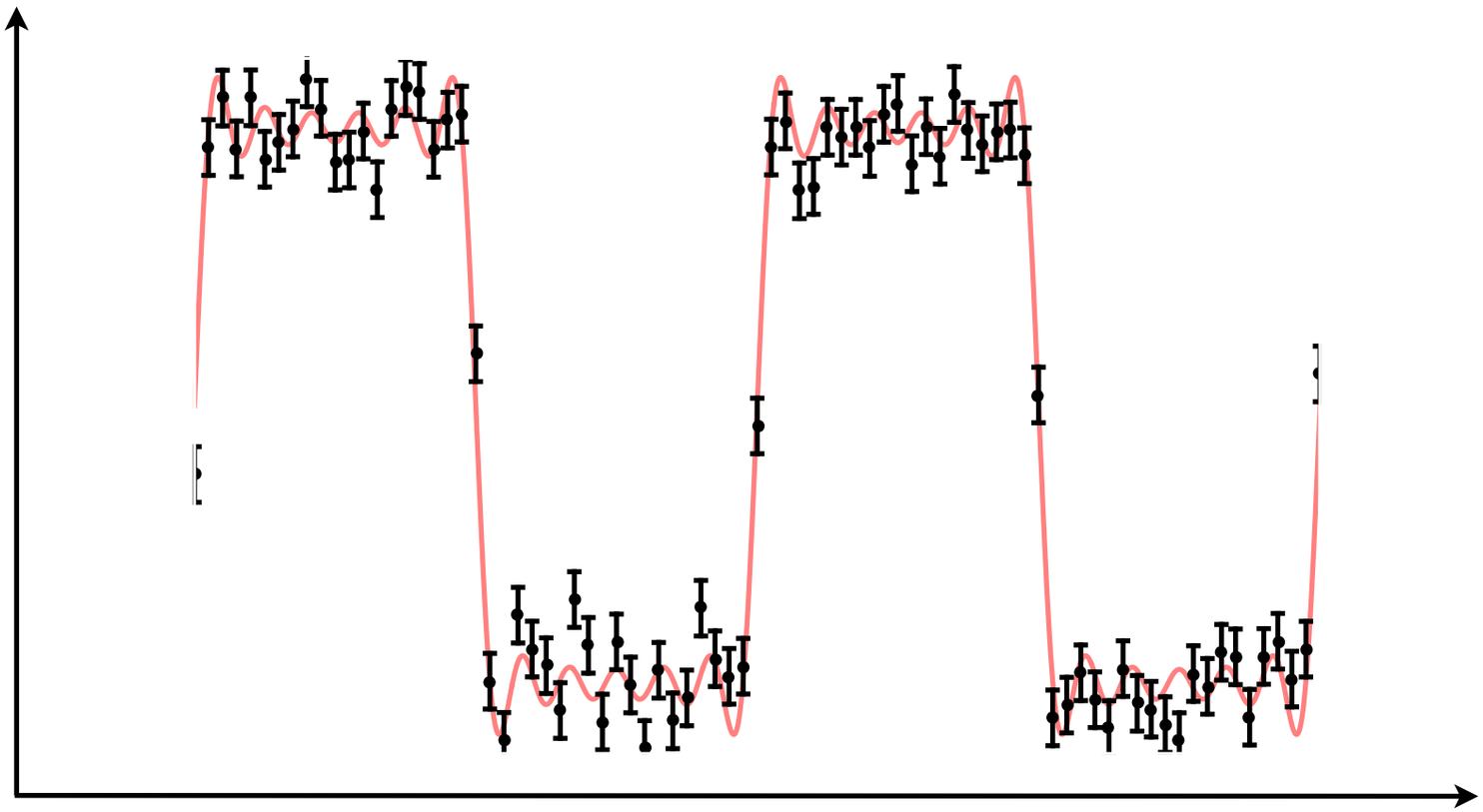
$$|\hat{f}(\omega)| = \sqrt{\mathrm{Re}[\hat{f}(\omega)]^2 + \mathrm{Im}[\hat{f}(\omega)]^2}$$


original function
$f(t)$
FT function
$|\hat{f}(\omega)|$

this is fine and nice, but in reality
we don't know f(t), we just have data of $\{t_i, f_i\}$



this is fine and nice, but in reality
we don't know f(t), we just have data of $\{t_i, f_i\}$



(and of course it's noisy too)

instead of doing a continuous FT...

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t)\exp(-2\pi \mathrm{i} t\omega)\,\mathrm{d}t$$

...we can also define a discrete FT (DFT)

$$\hat{f}_k = \sum_{m=0}^{(N-1)} f(t_{m+1})\exp\Big(-\frac{2\pi \mathrm{i} mk}{N}\Big)$$

where the period of the k^{th} DFT element is...

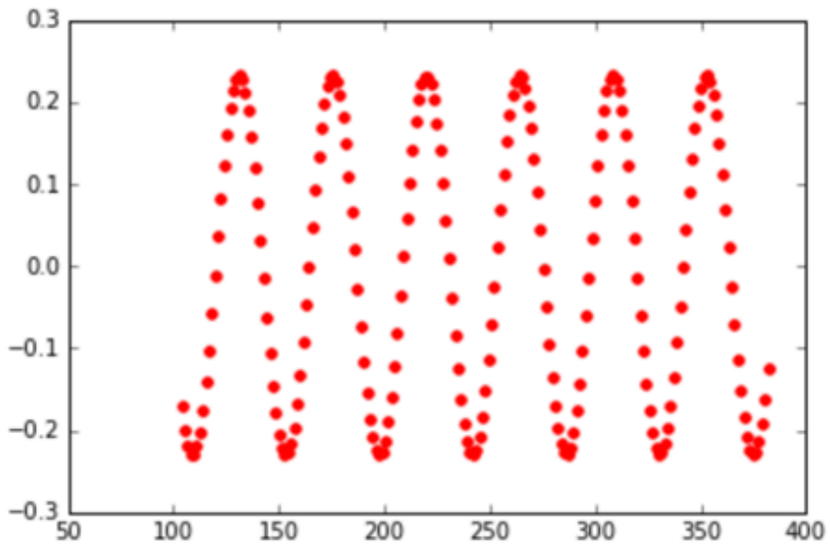
$$P_k = \frac{t_N - t_1}{k}$$

implement using **np.fft.fft(a)**

do not worry about the derivations or proofs here,
the concept is important to understand though

example data: randomly offset sine wave

In [149]:
```
import numpy as np
import matplotlib.pyplot as plt

# Generate some fake sine wave data
amp = 0.231
P = 44.319
phi = 1.761

tstart = 105.751
tend = 385.182
tstep = P/30.761
tlen = np.int(np.ceil((tend-tstart)/tstep))
t = [0 for i in range(tlen)]
y = [0 for i in range(tlen)]
for i in range(tlen):
    t[i] = tstart + (i-1.0)*tstep
    y[i] = amp*np.sin( 2.0*np.pi*t[i]/P + phi )

%matplotlib inline
plt.scatter(t,y,marker='o',color='r') # simple plot
plt.show()
```

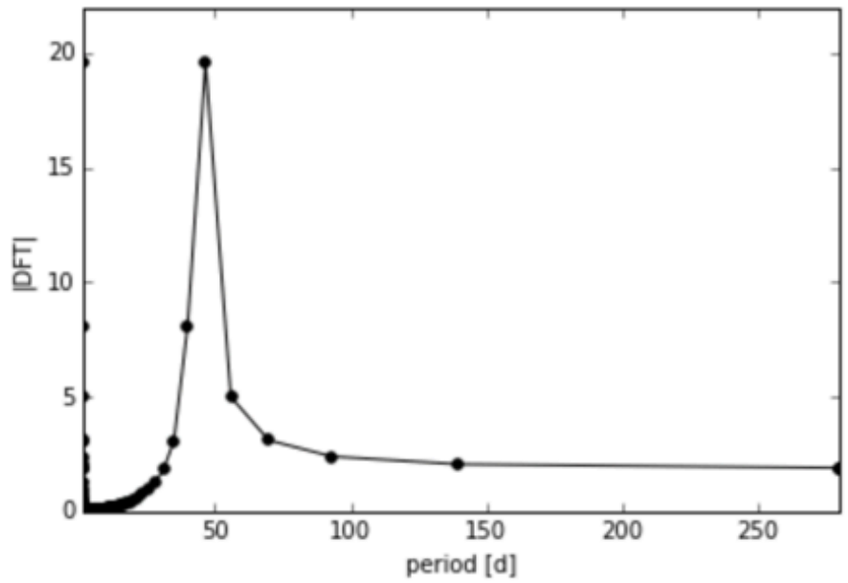


sine_data_dft.ipynb

the DFT has the same length as the original data, so it is therefore also sparse...

In [178]:
```
# Compute the Discrete Fourier Transform (DFT)
fft=np.abs(np.fft.fft(y))

# Compute the period at each DFT element
s = [i for i in range(tlen)]
Ptab = [(tend-tstart) for i in range(tlen)]
for i in range(1,tlen):
    Ptab[i] = (tend-tstart)/s[i]
```

In [221]:
```
%matplotlib inline
plt.plot(Ptab,fft,color='k')
plt.scatter(Ptab,fft,color='k')
plt.axis([tstep, (tend-tstart), 0, 22])
plt.xlabel('period [d]')
plt.ylabel('|DFT|')
plt.show()
print 'Peak power occurs at ',Ptab[np.argmax(fft)],' days'
print '(true period = ',P,' days)'
```




```
Peak power occurs at  46.5718333333  days
(true period =  44.319  days)
```

sine_data_dft.ipynb

What you need to know...

- Fourier transforms decompose functions or discrete data into the constituent underlying frequencies

- In principle, this is great (+ v fast if you use the Fast Fourier Transform, FFT by Cooley & Tukey 1965) but there are major drawbacks which make it's use quite limited in practice

1] DFT formally requires regularly sampled data [no gaps, homogenous sampling]. Workarounds typically involve zero-padding or intepolation (e.g. NUFFT = Non-Uniform Fast Fourier Transform, Greengard & Lee 2004).

2] Many DFT implementations are unweighted, modifying for heteroscedastic data is possible though (DWT = Discrete Weighted Transforms, Crandall & Fagin1994) but has limited flexibility for handling different data distributions

In practice, then, DFTs are useful on perfect data (such as that from a simulation) or as a quick look at real data

A Periodogram of Every *Kepler* Target and a Common Artifact at ~80 minutes

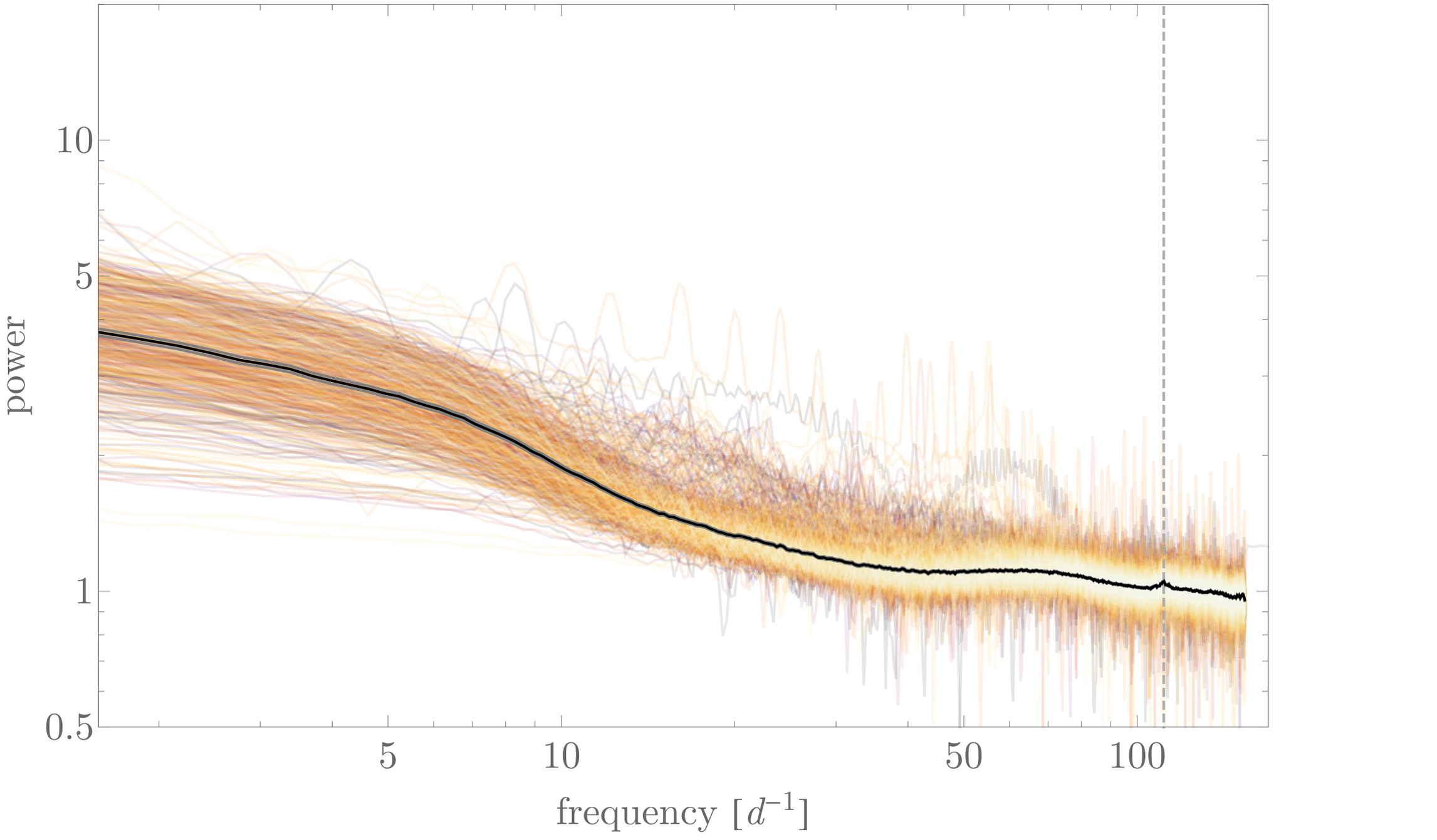
David Kipping[1]

Published May 2018 • © 2018. The American Astronomical Society. All rights reserved.

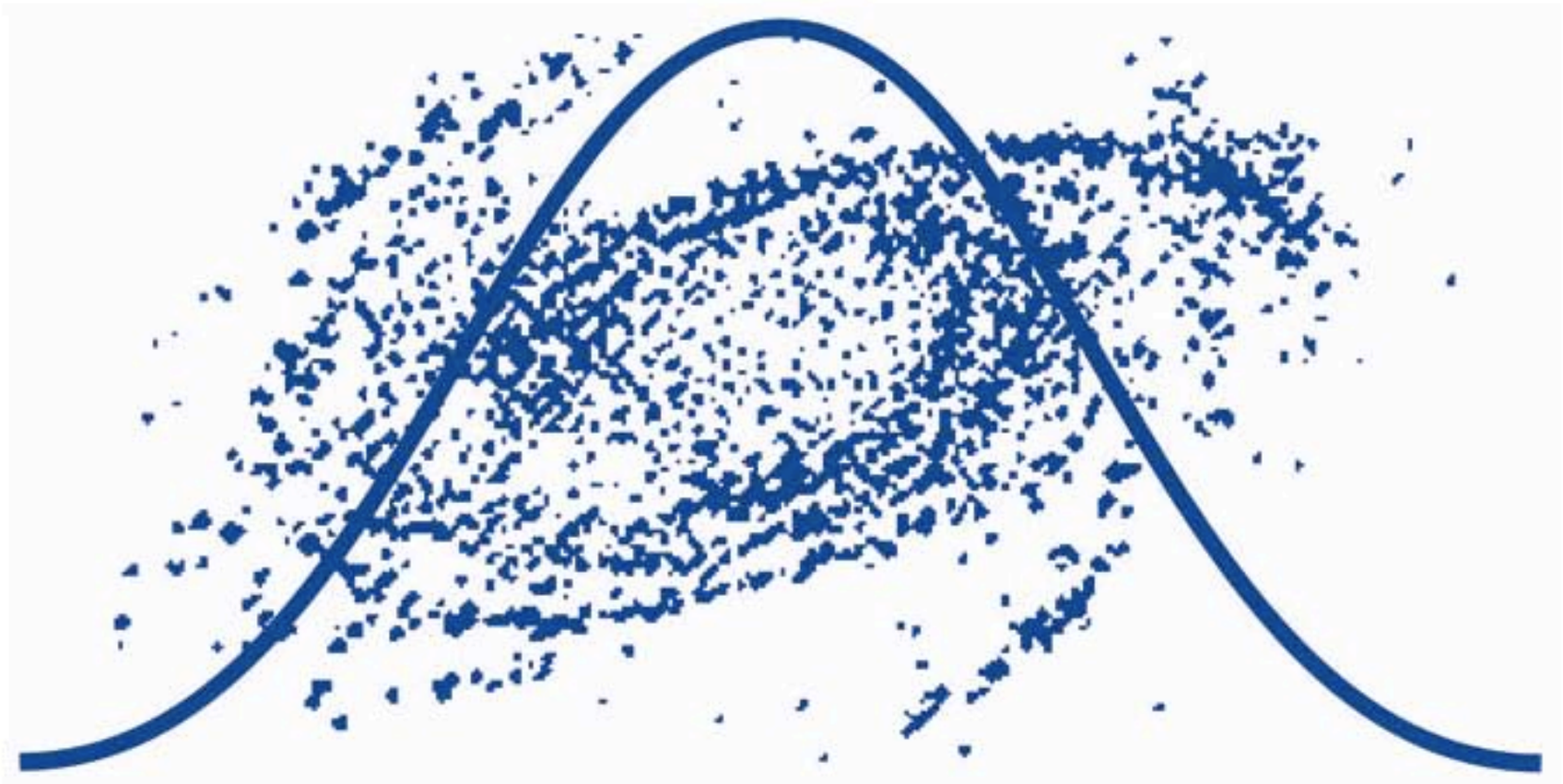
Research Notes of the AAS, Volume 2, Number 2

Citation David Kipping 2018 *Res. Notes AAS* **2** 15

DOI 10.3847/2515-5172/aac089



L8.2 Periodograms



Recall that for linear problems, we can solve for model parameters uniquely and quickly

let's say we want to model the data as being sinusoidal-like

$$f(t) = c + a\sin\left(\frac{2\pi t}{P} + \phi\right)$$

linear w.r.t. to c and a, but not phi

expand...

$$f(t) = c + a\left[\sin\left(\frac{2\pi t}{P}\right)\cos\left(\phi\right) + \cos\left(\frac{2\pi t}{P}\right)\sin\left(\phi\right)\right]$$

simplify...

$$f(t) = c + a\cos\phi\sin\left(\frac{2\pi t}{P}\right) + a\sin\phi\cos\left(\frac{2\pi t}{P}\right)$$

$$f(t) = c + a_c\sin\left(\frac{2\pi t}{P}\right) + a_s\cos\left(\frac{2\pi t}{P}\right)$$

now: linear w.r.t. to c, a_C, a_S [but not P]

$$f(t) = c + a_c \sin\left(\frac{2\pi t}{P}\right) + a_s \cos\left(\frac{2\pi t}{P}\right)$$

at each period, P, we can simply solve the normal equation...

$$\mathbf{X} = \begin{bmatrix} t_1^0 & \sin(2\pi t_1/P) & \cos(2\pi t_1/P) \\ t_2^0 & \sin(2\pi t_2/P) & \cos(2\pi t_2/P) \\ \vdots & \vdots & \vdots \\ t_n^0 & \sin(2\pi t_n/P) & \cos(2\pi t_n/P) \end{bmatrix} \qquad \mathbf{Y} = \begin{bmatrix} f(t_1) \\ f(t_2) \\ \vdots \\ f(t_n) \end{bmatrix}$$

the normal equation

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

$$\hat{\boldsymbol{\beta}} = \begin{bmatrix} c \\ a_c \\ a_s \end{bmatrix}$$

- this works for sparse data and is easy to do weighted version of

- at each period, plot $a=(a_c^2+a_s^2)^{1/2}$ or $\log\mathscr{L}/\Delta\chi^2$ improvement

- known as the Lomb-Scargle periodogram

scipy has a packaged implementation but not weighted :-(

scipy.signal.lombscargle

scipy.signal.lombscargle(*x*, *y*, *freqs*)

Computes the Lomb-Scargle periodogram.

The Lomb-Scargle periodogram was developed by Lomb [R205] and further extended by Scargle [R206] to find, and test the significance of weak periodic signals with uneven temporal sampling.

The computed periodogram is unnormalized, it takes the value `(A**2) * N/4` for a harmonic signal with amplitude A for sufficiently large N.

Parameters:

x *: array_like*
Sample times.

y *: array_like*
Measurement values.

freqs *: array_like*
Angular frequencies for output periodogram.

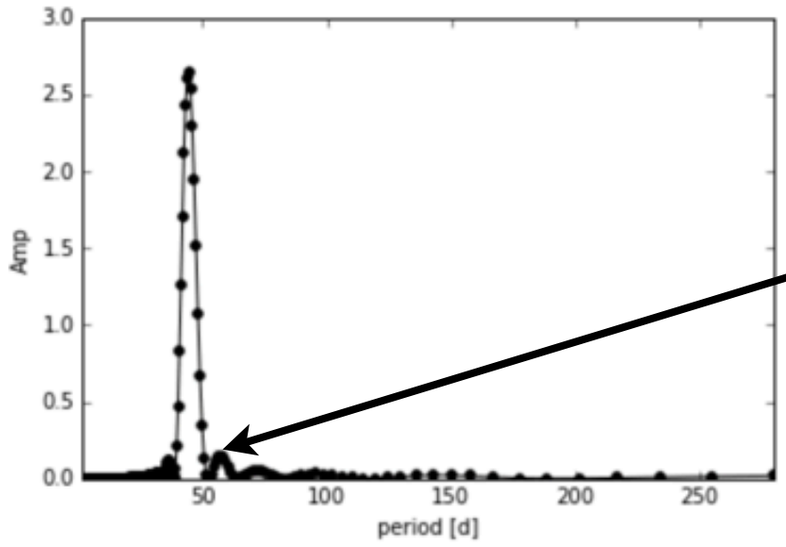
Returns:

pgram *: array_like*
Lomb-Scargle periodogram.

In [41]:
```
import scipy.signal as signal

# Define a frequency array to test
nfreq = 1000
freqmax = 1.0*np.pi/tstep
freqmin = 2.0*np.pi/(tend-tstart)
freqstep = (freqmax-freqmin)/nfreq

freqs = [0 for i in range(nfreq)]
periods = [0 for i in range(nfreq)]
for i in range(nfreq):
    freqs[i] = freqmin + i*freqstep
    periods[i] = 2.0*np.pi/freqs[i]

LSamp=signal.lombscargle(np.asarray(t), np.asarray(y), np.asarray(freqs))
```

have to use numpy arrays

In [42]:
```
%matplotlib inline

plt.scatter(periods,LSamp,color='k')
plt.plot(periods,LSamp,color='k')
plt.axis([tstep, (tend-tstart), 0, 3])
plt.xlabel('period [d]')
plt.ylabel('Amp')
plt.show()
print 'Peak power occurs at ',periods[np.argmax(LSamp)],' days'
print '(true period = ',P,' days)'
```



aliases

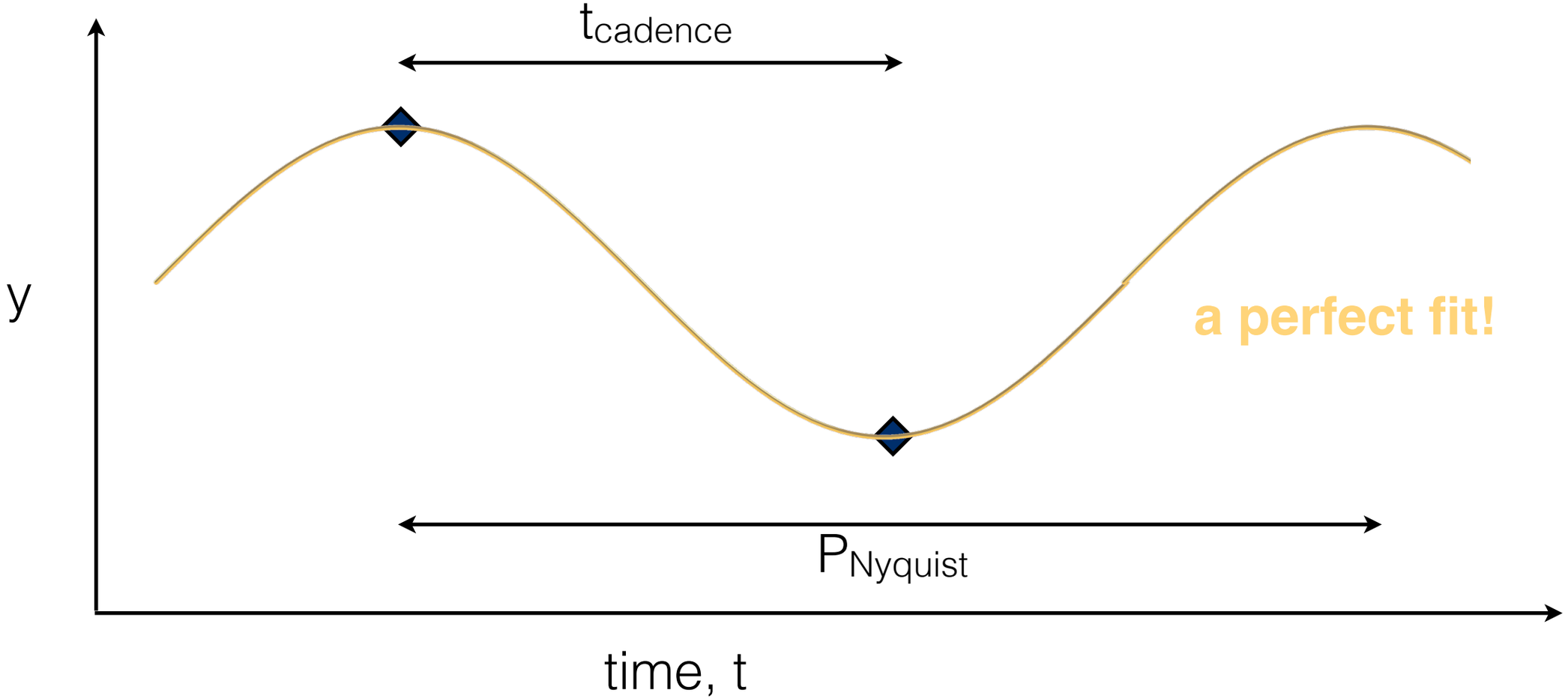
```
Peak power occurs at  44.5055358289  days
(true period =  44.319  days)
```

sine_data_dft.ipynb

things to be careful of...

- little point sampling frequencies higher than the Nyquist frequency

$P_{min} = 2\ t_{cadence}$ $f_{min} = f_{Nyquist} = ½\ f_{cadence}$

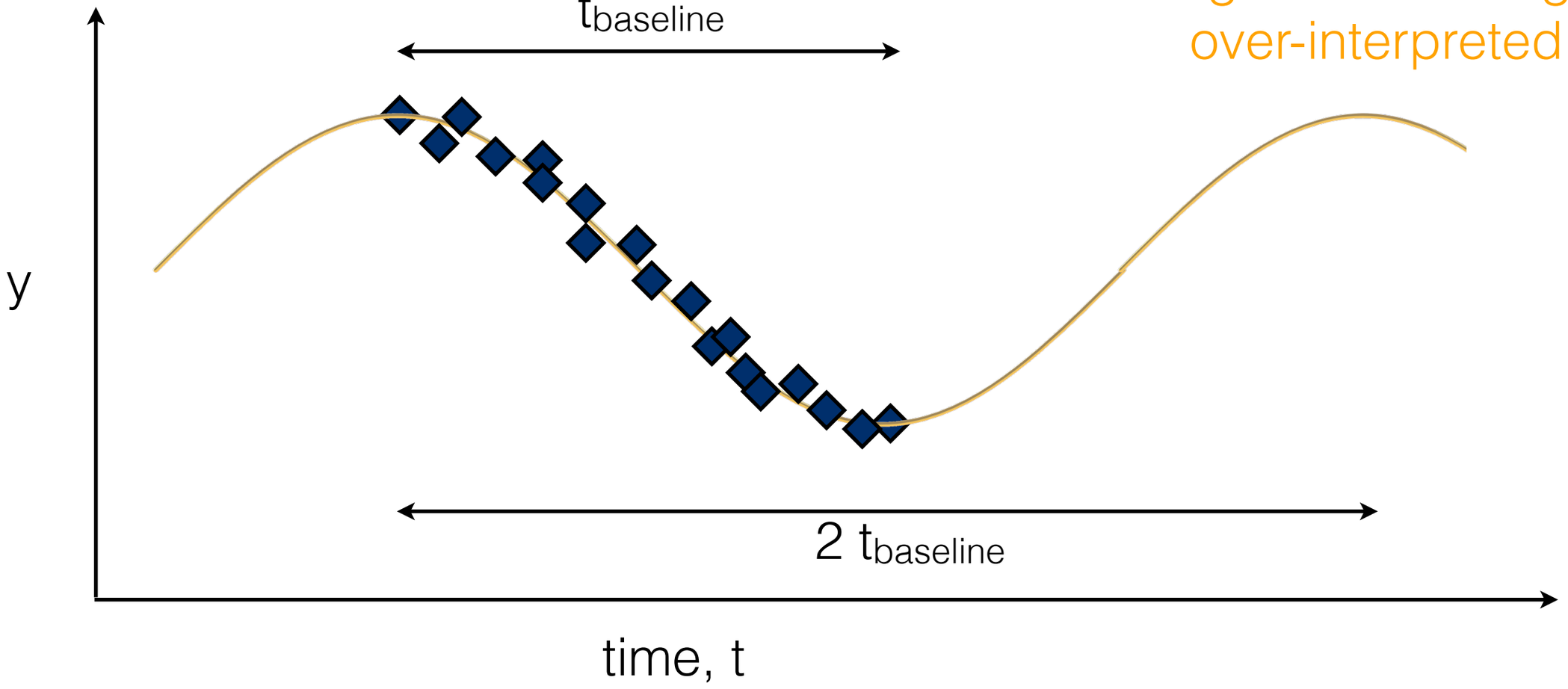


things to be careful of...

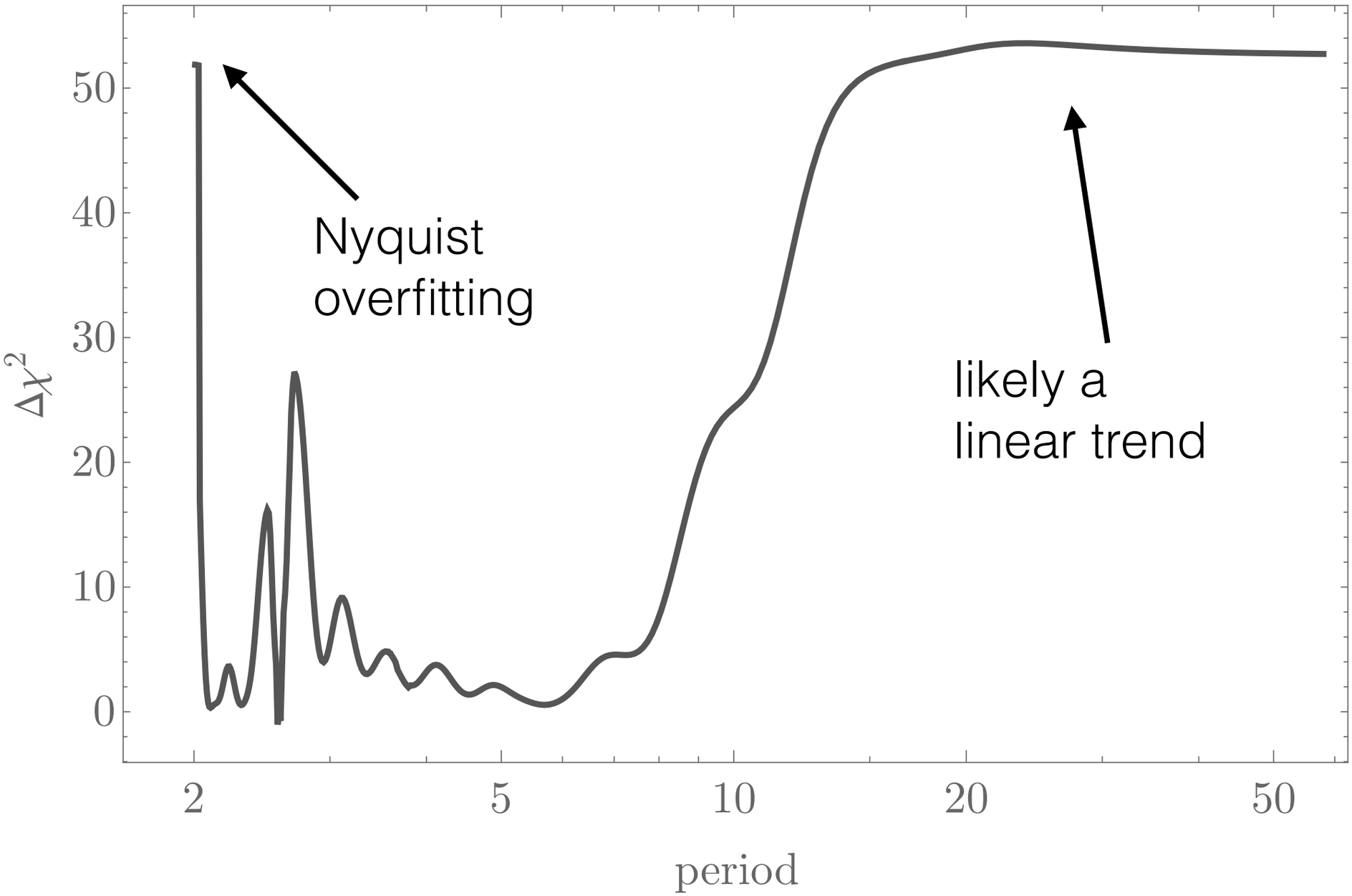
$f_{min} = f_{Nyquist} = ½\ f_{cadence}$

- little point sampling frequencies higher than the Nyquist frequency
- little point sampling periods longer than twice the baseline

$P_{max} = 2\ t_{baseline}$ $f_{min} = 1/(2\ t_{baseline})$



So when we see this sort of behavior….



summary

- Lomb-Scargle periodogram measures the best-fitting sinusoidal amplitude at input frequencies/periods

- Just make sure you use an appropriate range of frequencies

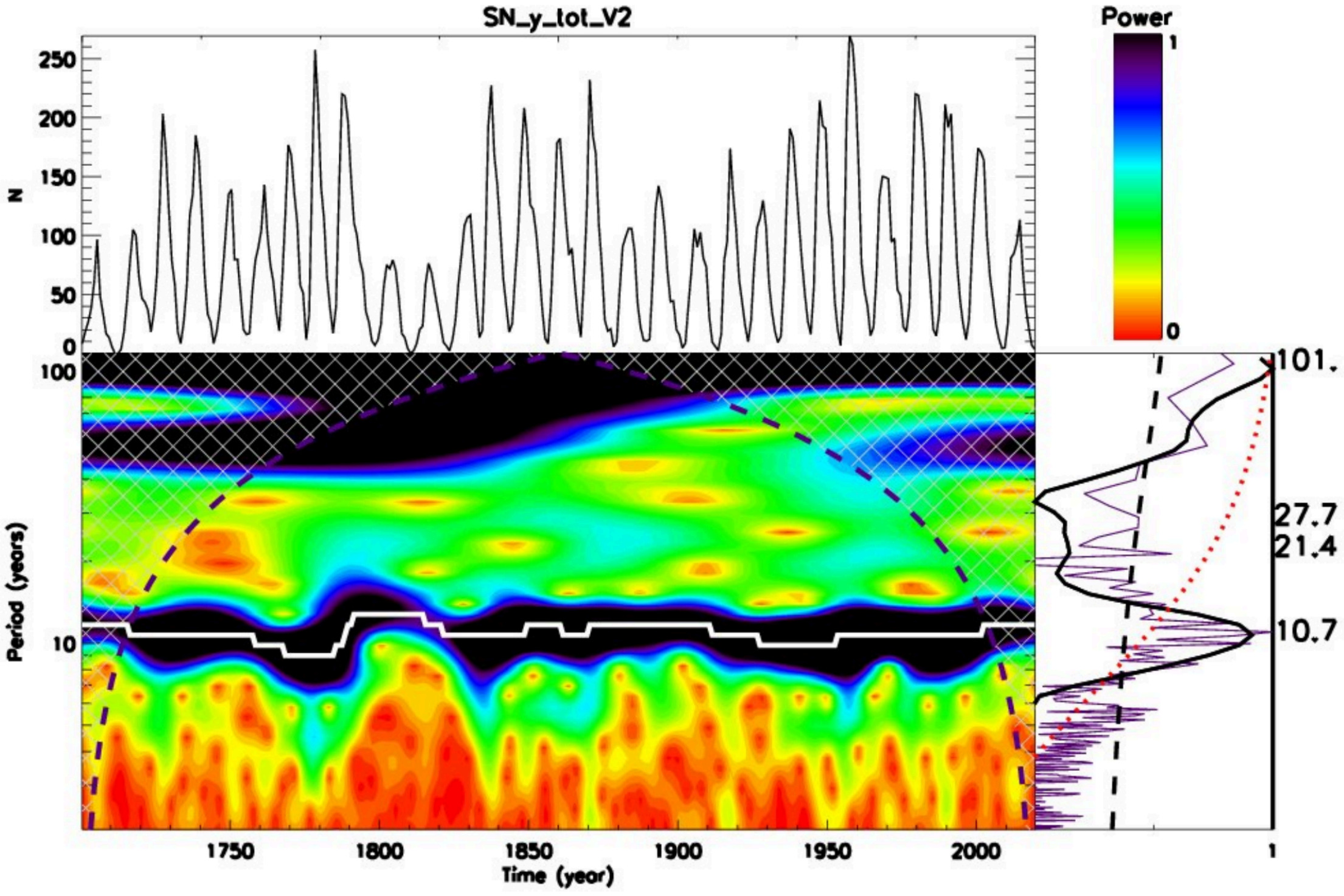
- Significance can be tested by bootstrapping of pure noise and measuring the distribution of max amplitudes detected

- The more frequencies you try, the more likely you are to find power!

You can use LS periodogram to identify important periodicities and remove these periodic trends from your data

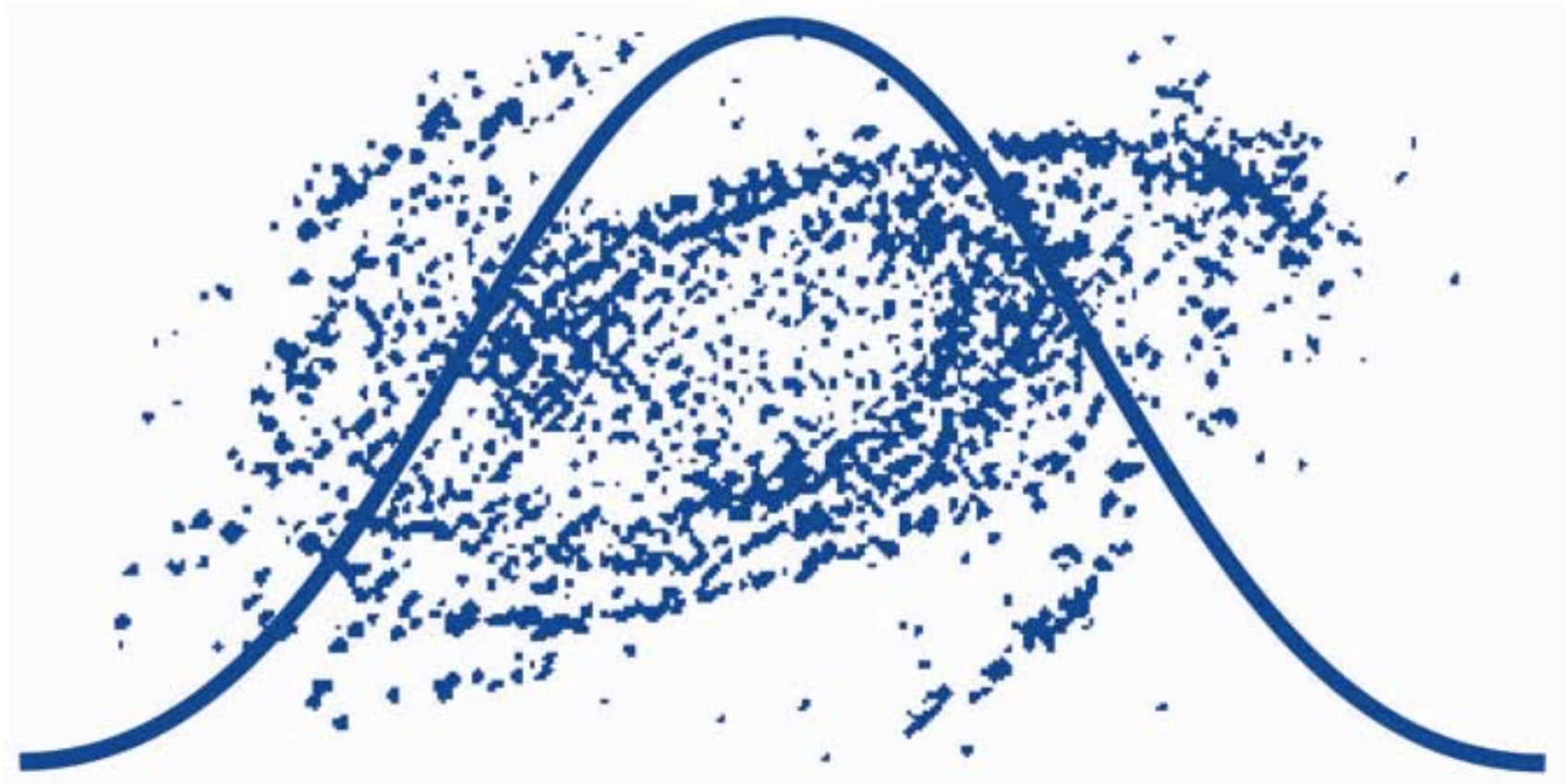
Deeper dive: “Understanding the Lomb Scargle periodogram”
https://arxiv.org/abs/1703.09824

We don’t discuss in depth, but, it’s worth noting that sometimes your periodicity is not stable over time.



In this case, the period needs to evolve. This is often visualized with spectrograms, either a sliding LS periodogram or via wavelet analysis

L8.3 Testing for Extra Noise



given some noisy data with known errors, is there a way of testing if there is excess power/noise indicative of unmodeled noise?

there are many tests and we won't go into them all...

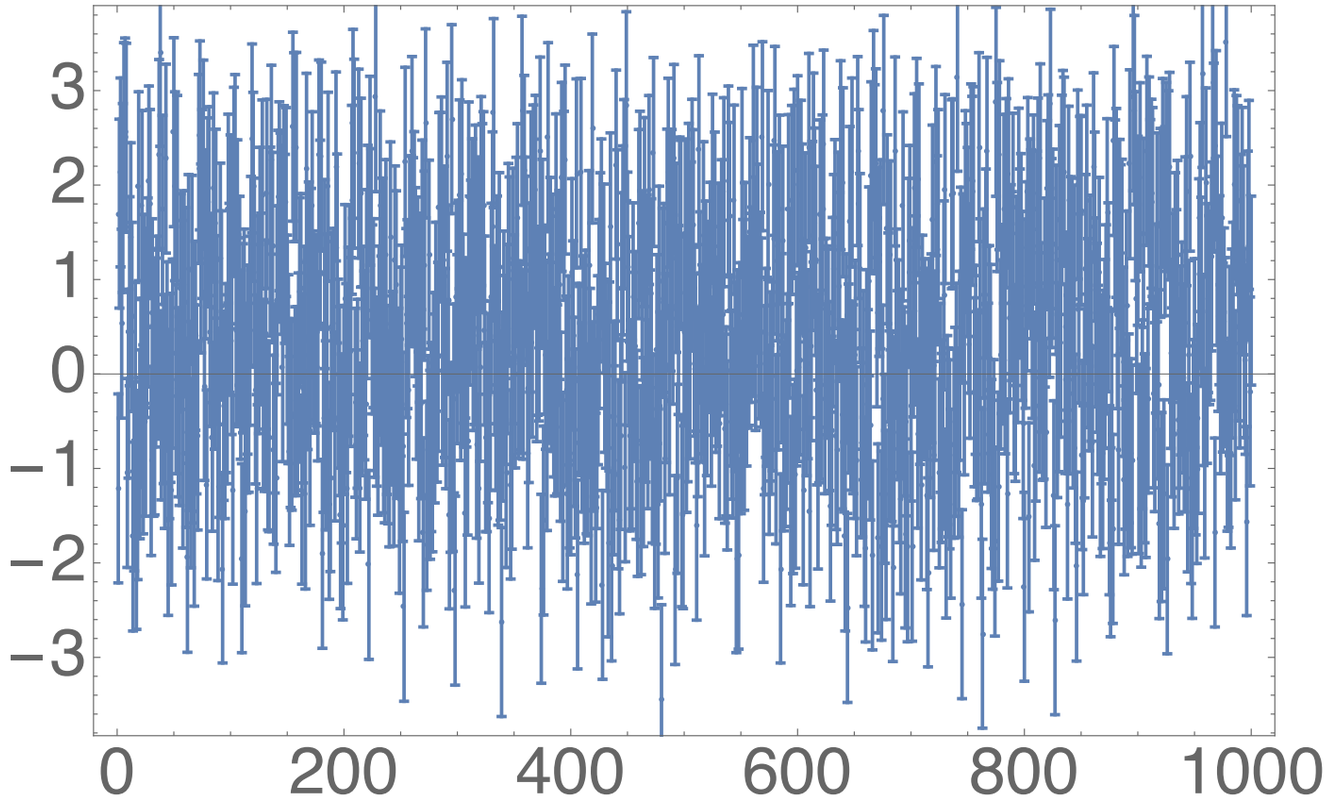
autocorrelation based tests

chi2 excess

allan variance

binning behaviour

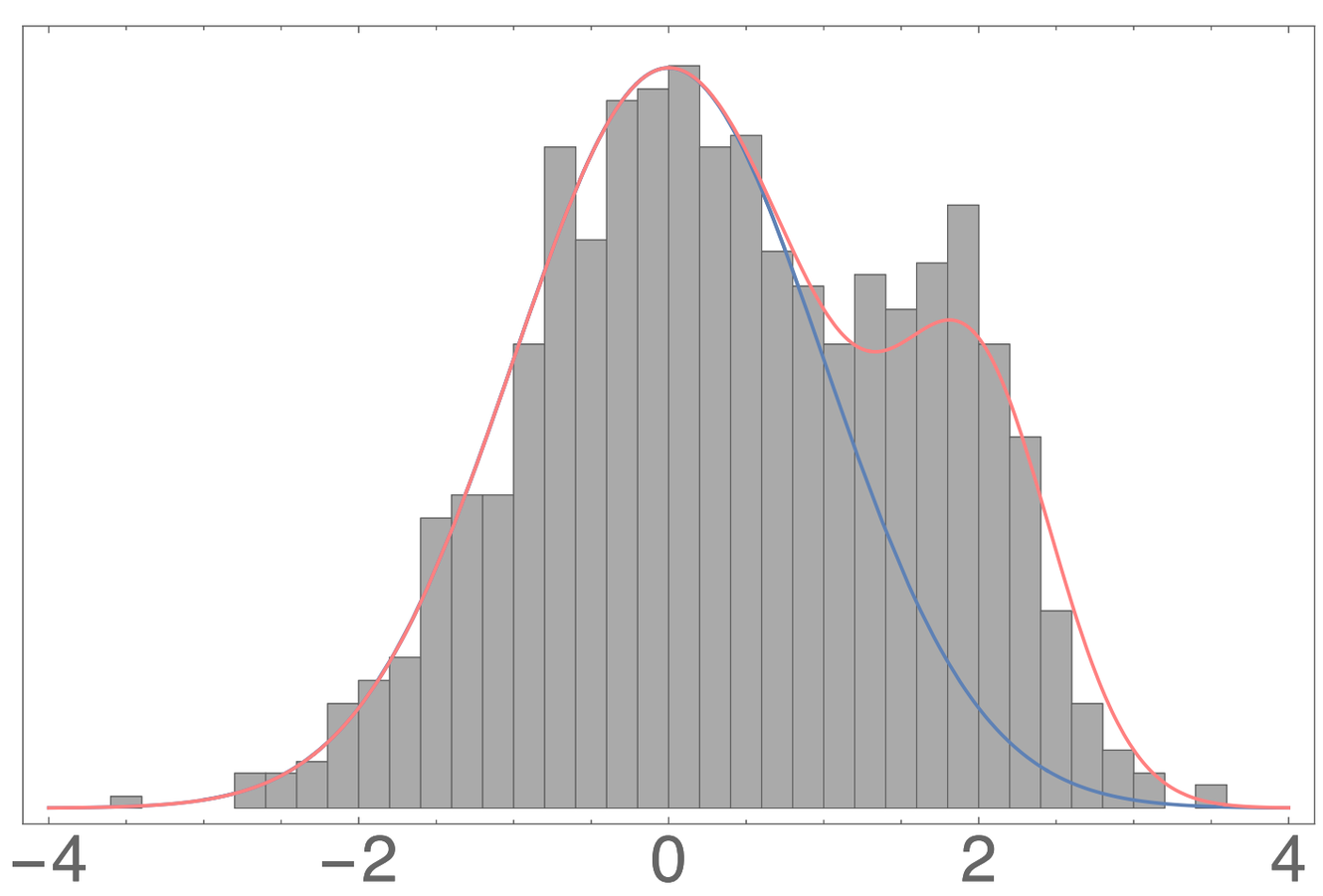
normalized histograms



difficult to tell by eye if data have excess noise or not

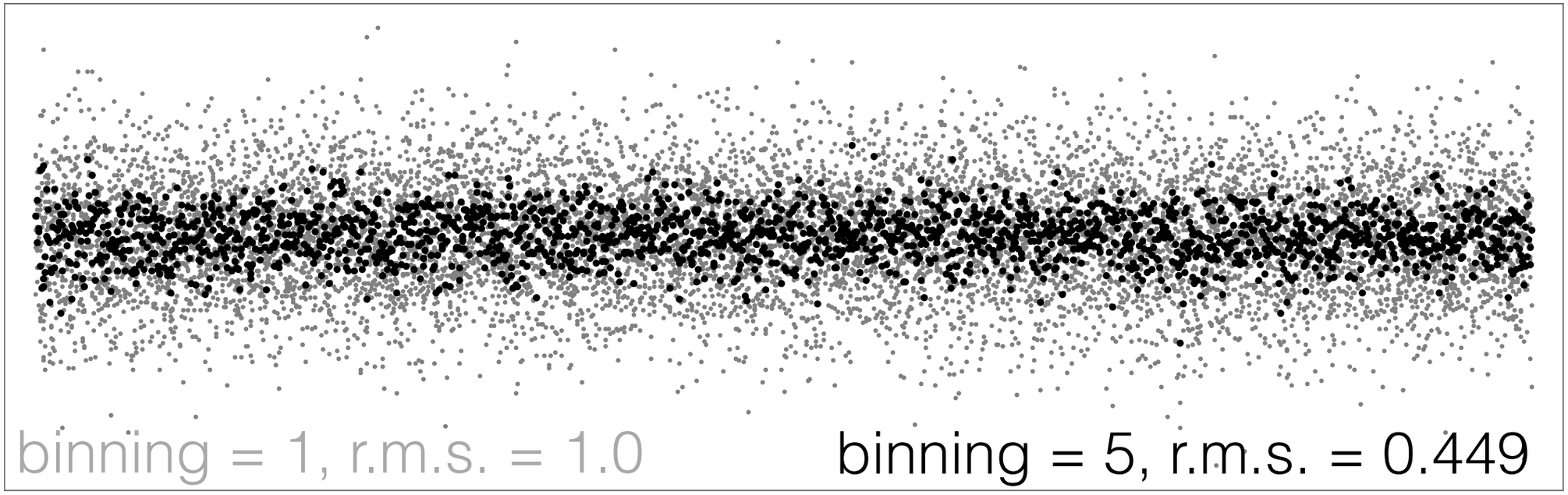
chi2 = 1640
for 1000 data points

integrating the pdf of a chi2 distribution => 10^{-33} prob (**p-value**)

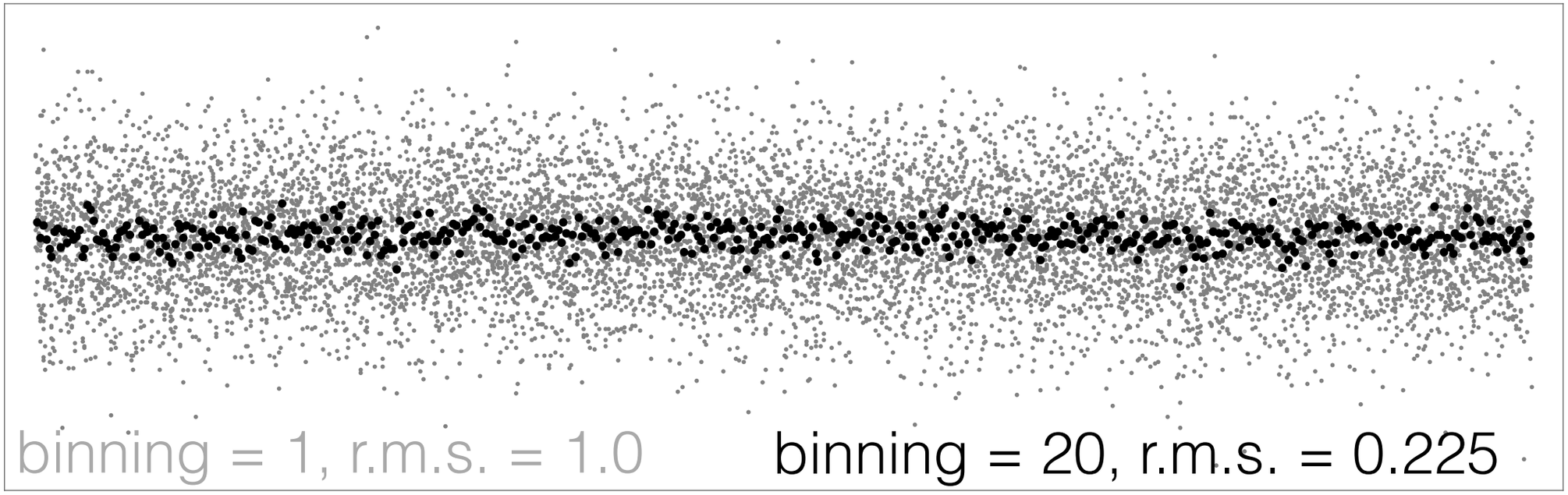


making a histogram of the normalized residuals gives more information about what is causing this

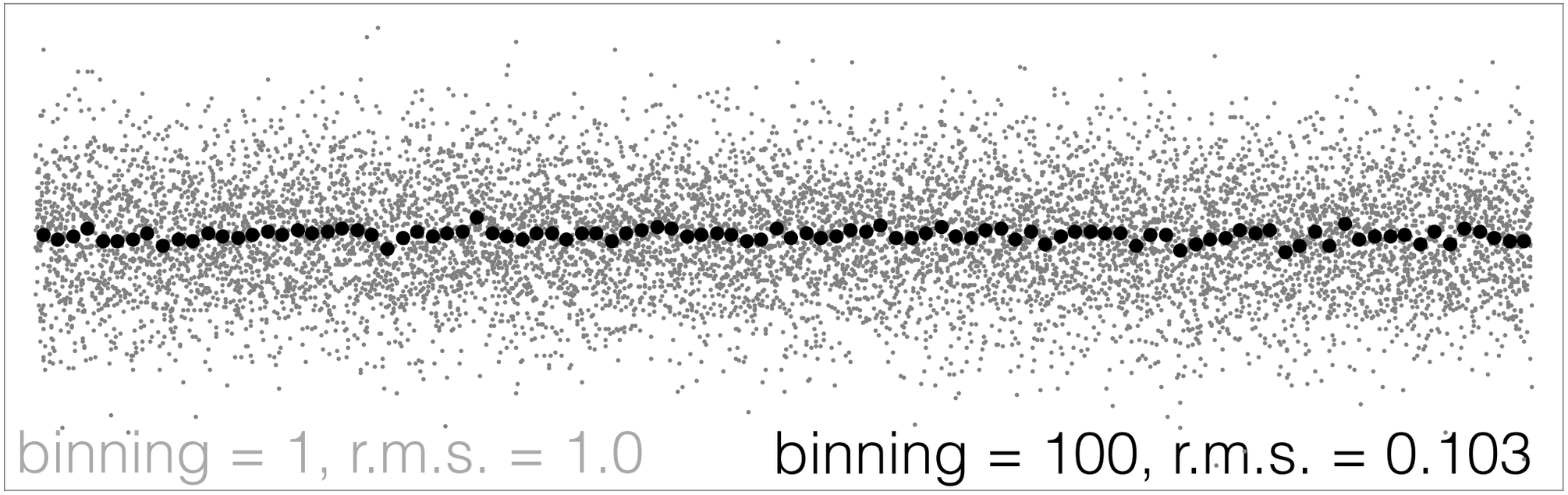
one of the most popular ways of investigating excess noise in time series analysis is via binning



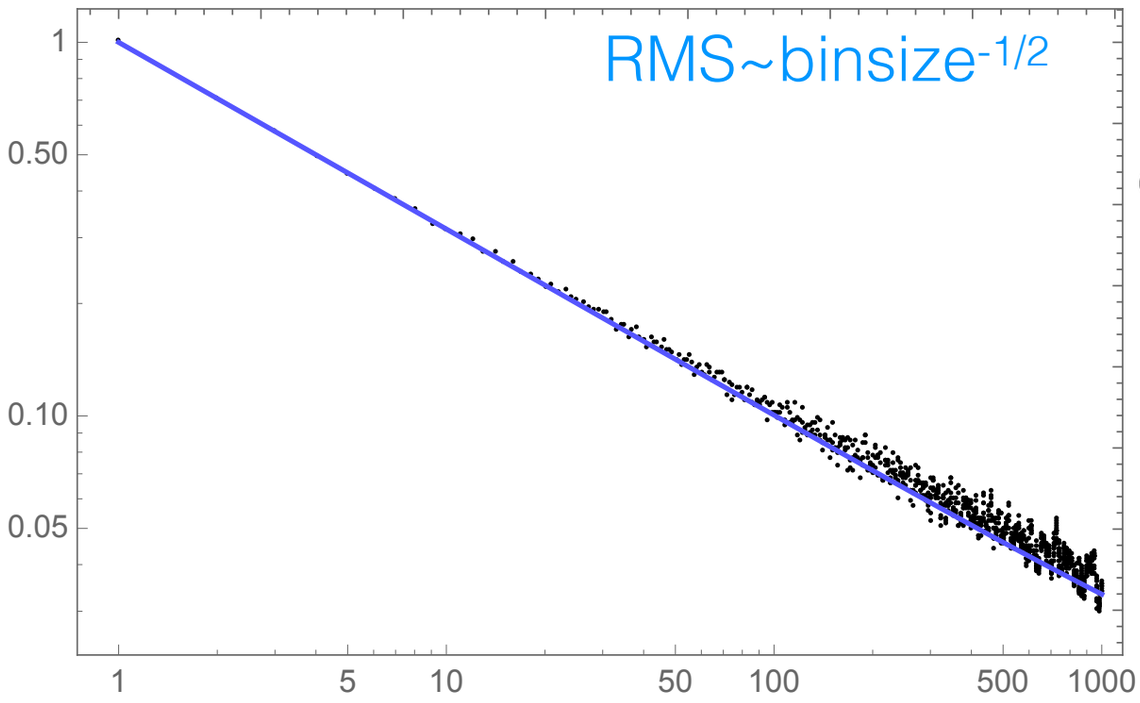
one of the most popular ways of investigating excess noise in time series analysis is via binning



one of the most popular ways of investigating excess noise in time series analysis is via binning



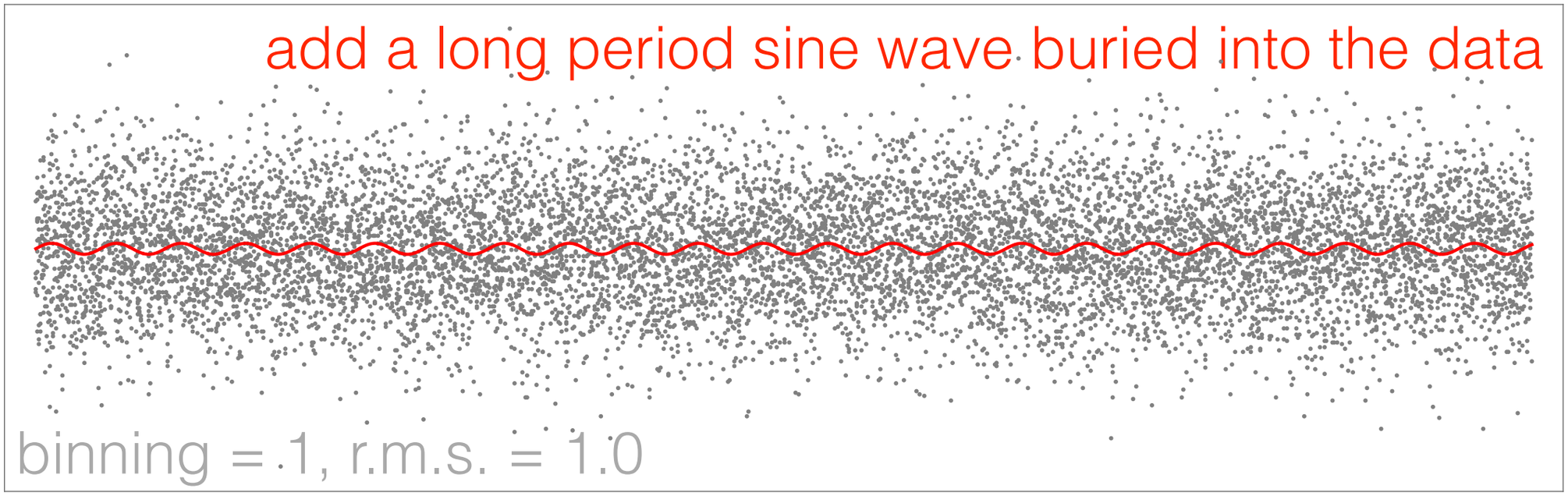
a “clean” time series



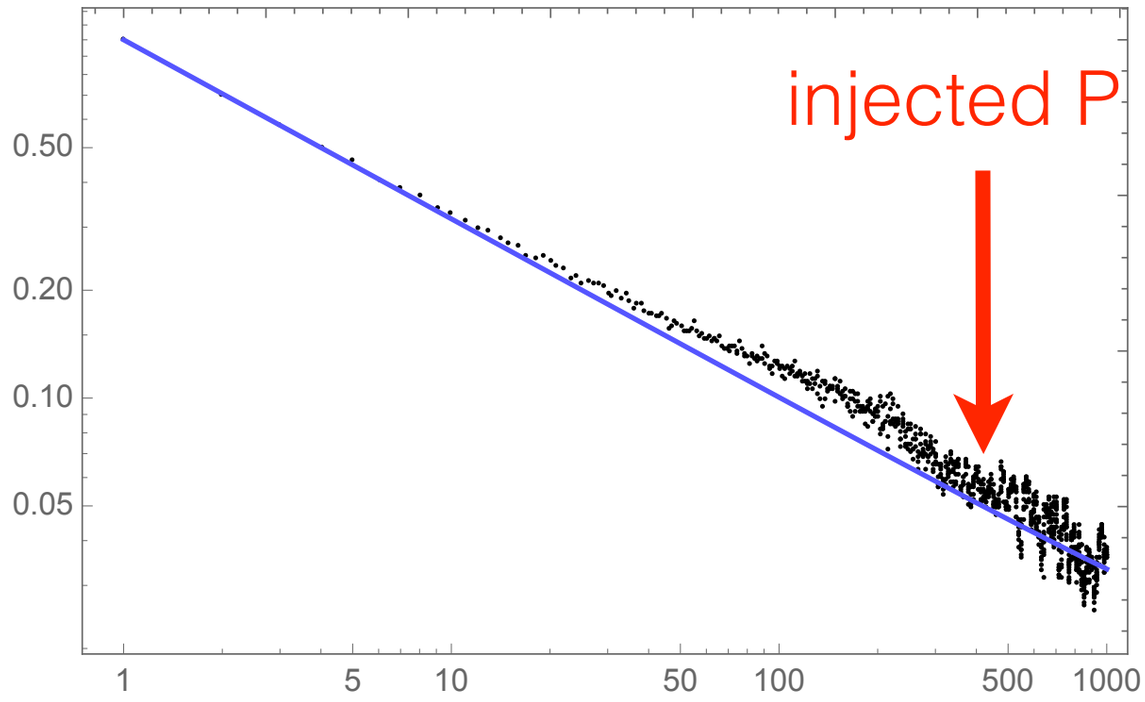
bin size # of bins

$$\sigma_N = \sigma_1 N^{-1/2}[M/(M-1)]^{-1/2}$$

one of the most popular ways of investigating excess noise in time series analysis is via binning



binned rms deviates

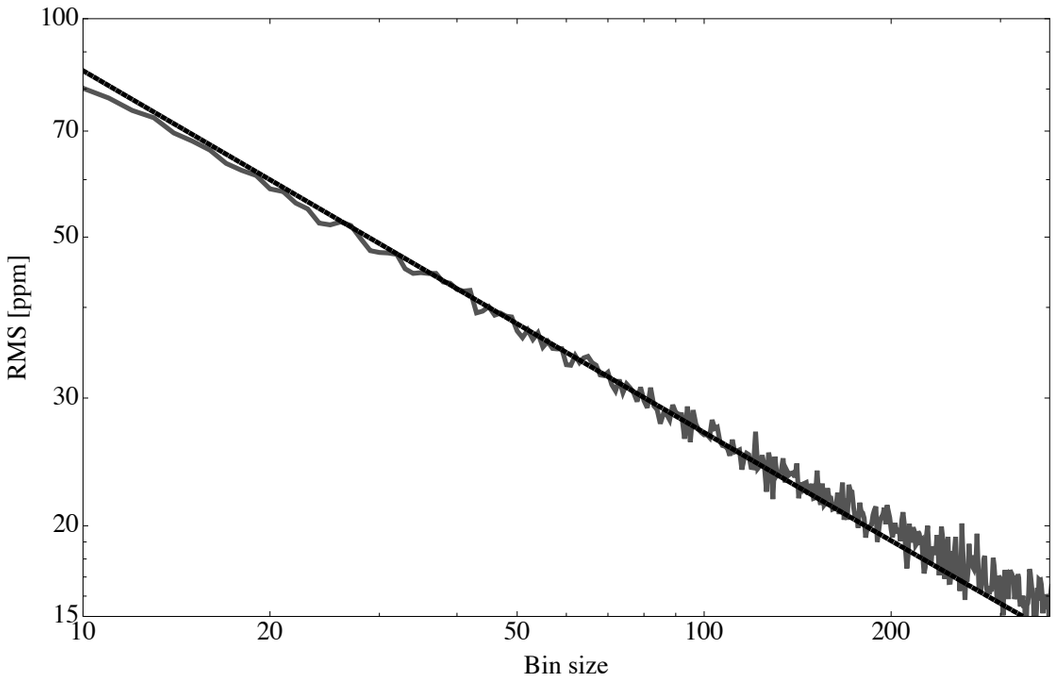


bin size # of bins

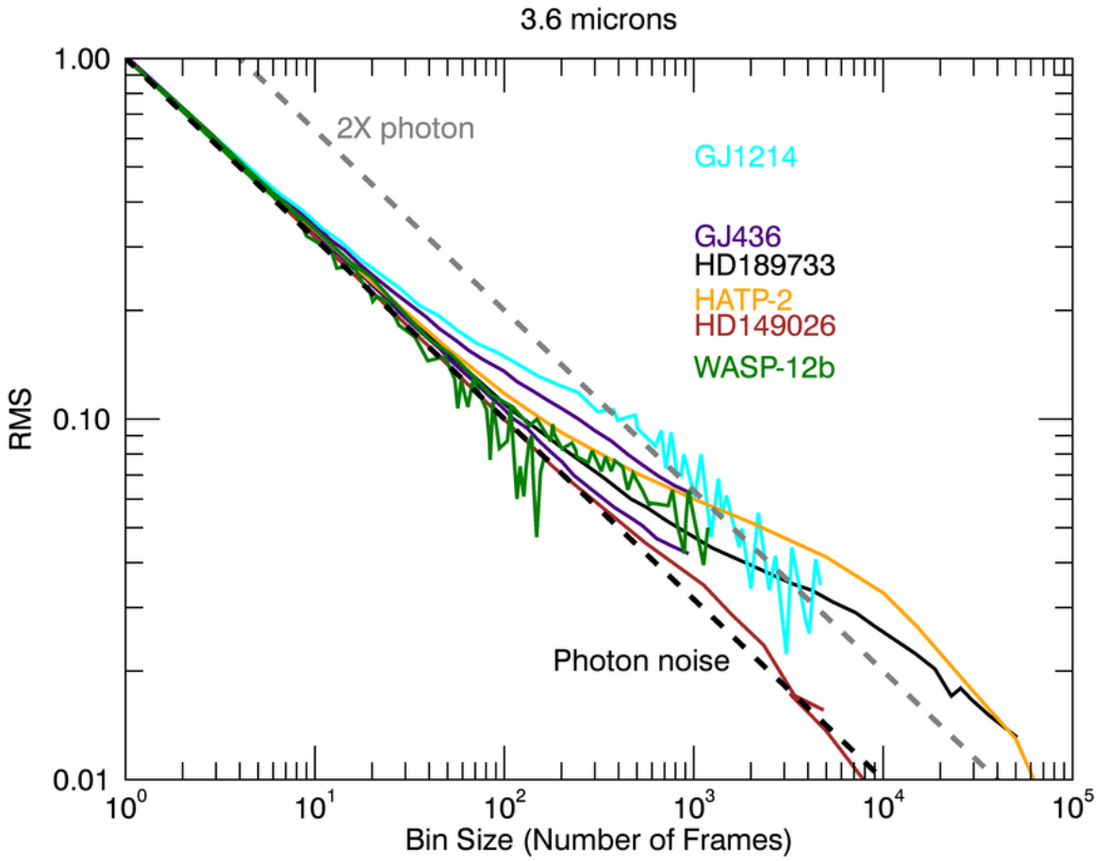
$$\sigma_N = \sigma_1 N^{-1/2}[M/(M-1)]^{1/2}$$

difficult to infer actual period this way, but could now be motivated to use a periodogram

one of the most popular ways of investigating excess noise in time series analysis is via binning



Kepler



Spitzer

quick example, try yourself!

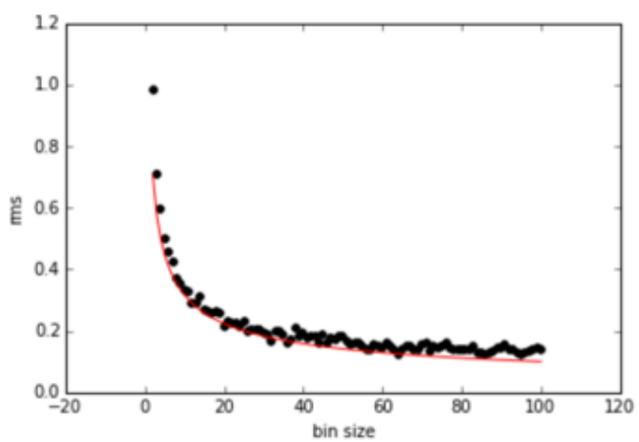
jupyter binning (unsaved changes)

File Edit View Insert Cell Kernel Help | Python 2

Code CellToolbar

In [17]:
```
import numpy as np
import matplotlib.pyplot as plt

# Generate some fake data
data = np.random.normal(0.0, 1.0, 1000)

bmax = 100 # Maximum bin size to try
binsize = [0 for i in range(2,bmax+1)] # Initialize binsize array
rms = [0 for i in range(2,bmax+1)] # Initialize rms array
rmstheory = [0 for i in range(2,bmax+1)] # Initialize rmstheory array

for b in range(2,bmax+1):
    # Create a binned time series of the data = "binned"
    imax = np.int( np.floor(len(data)/b) ) # Length of the new binned array
    binned = [0 for i in range(imax)]      # Initialize this new array
    for i in range(imax):
        binned[i] = np.mean(data[b*i+1:(i+1)*b])
    # Compute the r.m.s. of this binned time series
    rms[b-2] = np.std(binned)
    binsize[b-2] = b
    rmstheory[b-2] = b**(-0.5)*(imax/(imax-1))**(0.5)
```

In [20]:
```
%matplotlib inline
plt.scatter(binsize,rms,color='k')
plt.plot(binsize,rmstheory,color='r')
plt.xlabel('bin size')
plt.ylabel('rms')
plt.show()
```



binning.ipynb